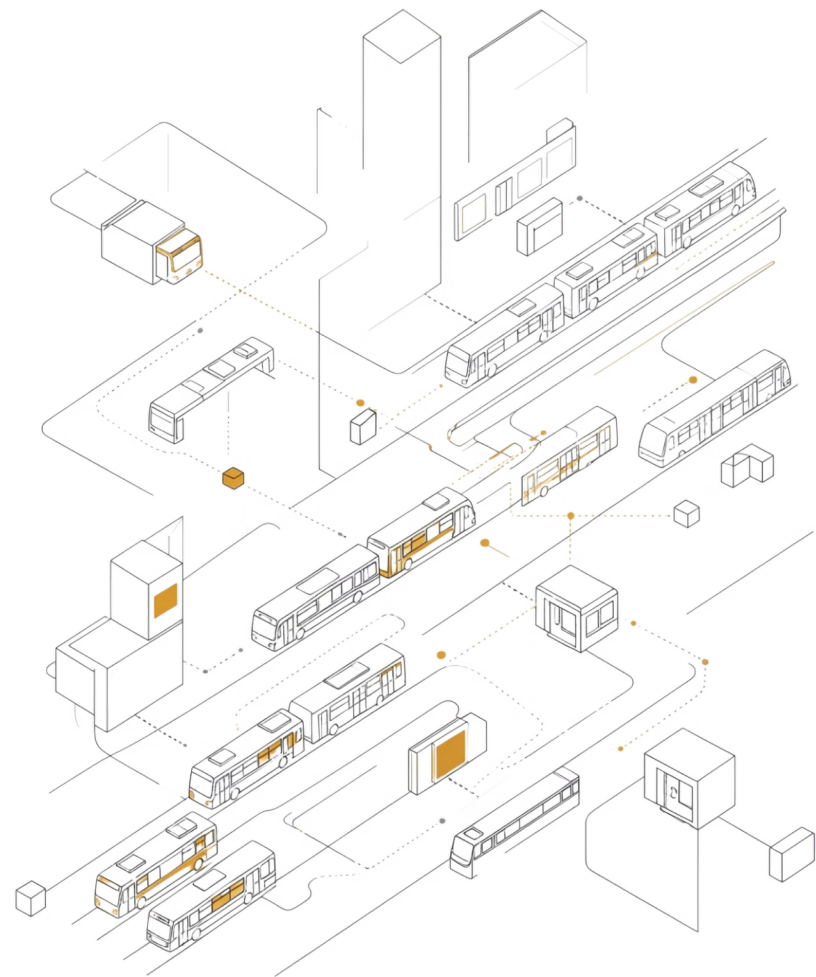
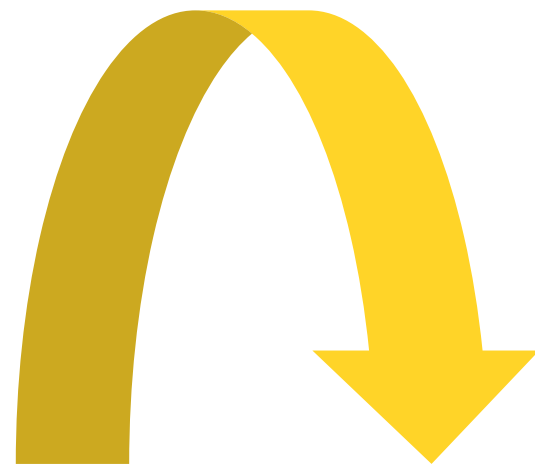




#da9b4
#EBCC575 FDF4DB

Smart City Transport System

Database Lab Semester Project

Presented by

Usman Ali Shahid (24F-0512)

Taheer Bin Hussain (24F-0567)

FAST-NUCES, Faisalabad Campus

Project Objective

Core Mission

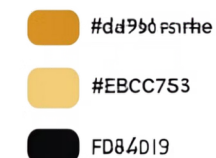
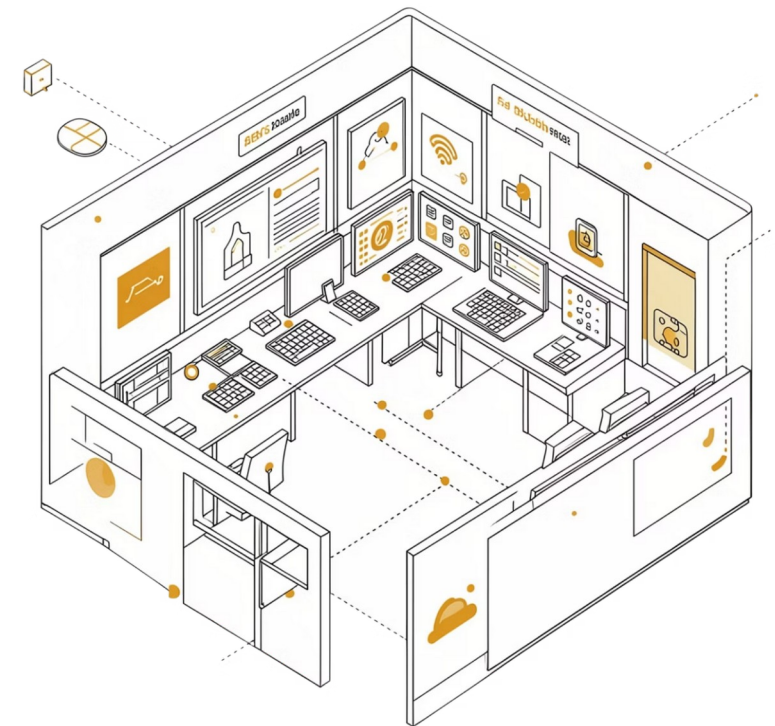
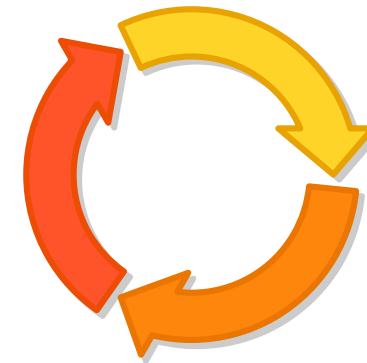
Implement advanced database concepts to develop a practical, real-world public transport management system that tracks operations across the entire transit network.

Seamless Coordination

Ensure seamless coordination between stations, vehicles, schedules, and ticket sales — all managed through a unified relational database.

Key Focus Areas

- Enforcing strict data integrity using PL/SQL constraints
- Resolving complex entity relationships
- Automating business logic with triggers and procedures



List of Tables with Attributes

Station

*stationID (PK), stationName,
location, platform*

Passenger

*passengerID (PK),
passengerName,
passengerEmail,
phoneNumber, DOB, age*

Vehicle (Superclass)

*vehicleID (PK), capacity,
model*

Bus (Subclass):
busNumber, busType

Train (Subclass):
*trainNumber,
numberOfCoaches*

Schedule

*scheduleID (PK),
arrivalTime, departureTime,
routeID (FK), vehicleID (FK),
driverID (FK)*

Route

*routeID (PK), routeName,
distance, startStationID (FK),
endStationID (FK)*

Driver

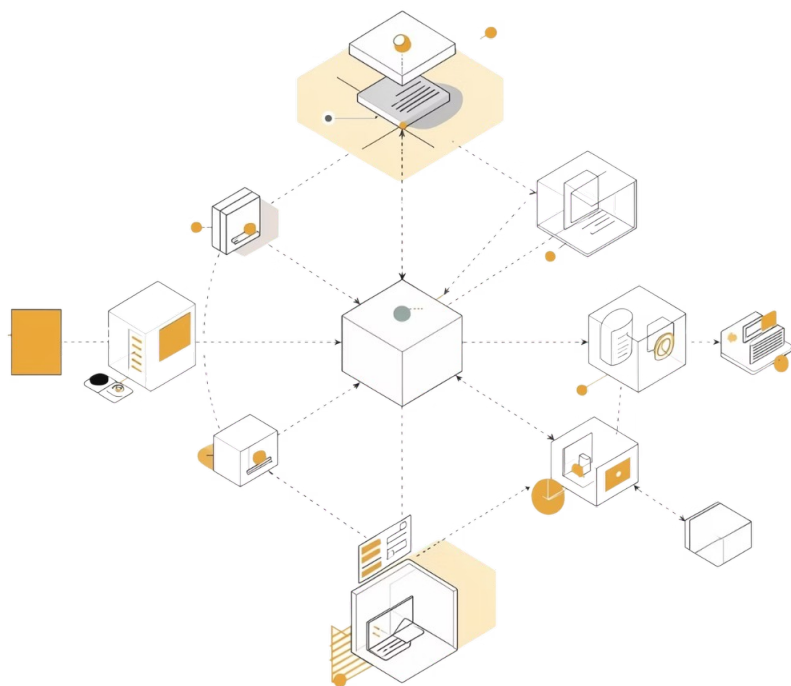
*driverID (PK), driverName,
licenseNumber, experience*

Ticket

*ticketNumber (PK), price, type, issueDate, passengerID (FK),
scheduleID (FK)*



Enhanced ER Diagram (EERD) & Relationship



Relationships — Cardinality & Participation					
Relationship	Entity 1	Cardinality	Entity 2	Type	Participation
buys	PASSENGER	1 : N	TICKET	Identifying	Total (Ticket)
valid for	TICKET	N : 1	ROUTE	Regular	Total (Ticket)
includes	ROUTE	M : N	STATION	Regular	Partial (both)
follows	ROUTE	1 : N	SCHEDULE	Regular	Total (Schedule)
assigned to	VEHICLE	M : N	ROUTE	Regular	Partial (both)
drives	DRIVER	M : N	VEHICLE	Regular	Partial (both)
serves	STATION	M : N	SCHEDULE	Regular	Partial (both)
ISA	VEHICLE	1 : 1	BUS/TRAIN	Inheritance	Total (subclass)

Data Fetching — Subqueries

What Are Subqueries?

Subqueries are nested SQL statements used to isolate specific data points before executing the main query. They allow dynamic filtering based on computed values from the

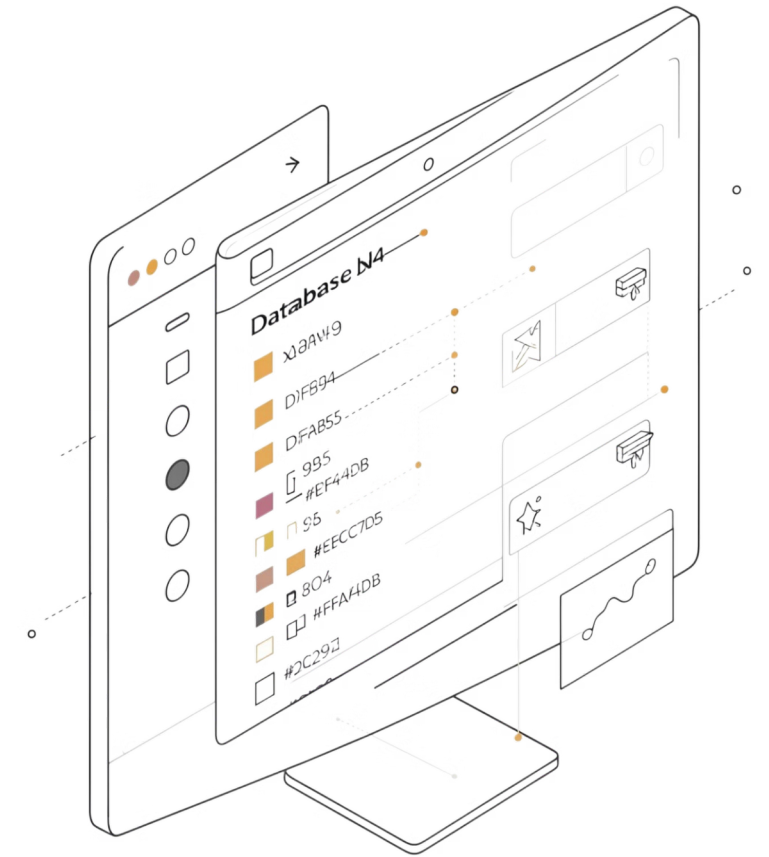


PASSENGERNAME	AGE
Hassan Raza	34
Bilal Sheikh	39
Zainab Ali	32
Ahmed Iqbal	36
Kamran Aslam	31
Fahad Malik	33
Imran Shah	37

SQL Implementation

```
SELECT passengerName,  
age  
FROM Passenger  
WHERE age > (  
    SELECT AVG(age)  
    FROM Passenger  
);
```

The inner query computes the average age across all passengers. The outer query then returns only those passengers whose age exceeds that computed average.

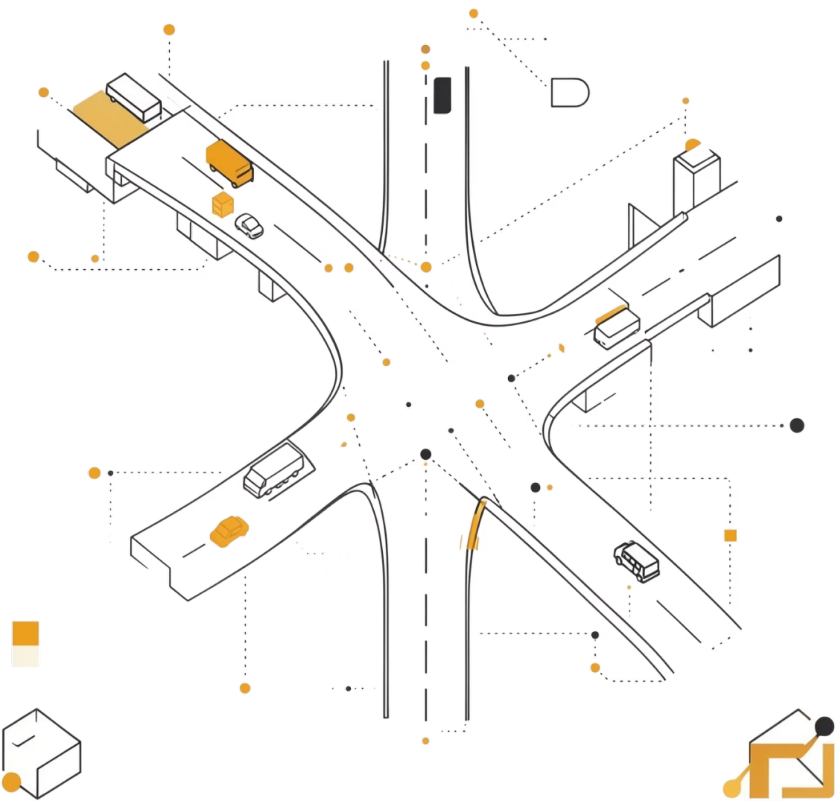


Data Fetching — Joins

What Are Joins?

Joins combine relational data from multiple tables to create comprehensive, human-readable reports. Instead of raw foreign key IDs, joins resolve them into meaningful text values.

 **Example Use Case:** Generating full route details by fetching the actual names of start and end stations instead of just their numeric ID numbers.



```
SELECT r.routeName,  
       s1.stationName AS Start,  
       s2.stationName AS End,  
       r.distance  
FROM Route r  
JOIN Station s1  
  ON r.startStationID = s1.stationID  
JOIN Station s2  
  ON r.endStationID = s2.stationID;
```

ROUTENAME	STARTSTATION	ENDSTATION	DISTANCE
Gujranwala-Lahore Circle	Gujranwala Point	Lahore Junction	70
Faisalabad-Lahore Express	Faisalabad Central	Lahore Junction	120
Peshawar-Islamabad Express	Peshawar Central	Islamabad Metro	180
Lahore-Islamabad Highway	Lahore Junction	Islamabad Metro	375
Quetta-Multan Long Route	Quetta Station	Multan Station	950
Karachi-Multan Route	Karachi Terminal	Multan Station	850
Islamabad-Rawalpindi Local	Islamabad Metro	Rawalpindi Hub	15
Faisalabad-Sialkot Link	Faisalabad Central	Sialkot Stop	95

Data Fetching — Views

What Are Views?

Views create virtual tables that simplify complex analytics for administrative users. They eliminate the need to repeatedly write complex joins by encapsulating the logic into a reusable named query.

- ✓ **Example Use Case:** A dashboard view showing only the highly experienced senior drivers — those with more than 10 years of service — for quick administrative reference.

Views improve security by exposing only the columns and rows that administrators need, without granting access to the full underlying tables.

SQL Implementation

```
CREATE OR REPLACE VIEW
  SeniorDrivers_VW
AS
  SELECT
    driverName,
    licenseNumber,
    experience
  FROM Driver
  WHERE experience >= 12;
```

DRIVERNAME	LICENSENUMBER	EXPERIENCE
Rashid Mahmood	LIC-2012-002	12
Naveed Akhtar	LIC-2010-004	14
Shahid Iqbal	LIC-2011-009	13

- 📄 Once created, administrators can simply run `SELECT * FROM SeniorDrivers_VW;` to instantly retrieve the filtered result set.

Data Fetching — Triggers

What Are Triggers?

PL/SQL Triggers act as automated tripwires that fire automatically on **INSERT** or **UPDATE** events to enforce strict business rules — without requiring any manual intervention from the application layer.



Example Use Case:

Preventing data entry errors by blocking negative ticket prices from ever being committed to the database.

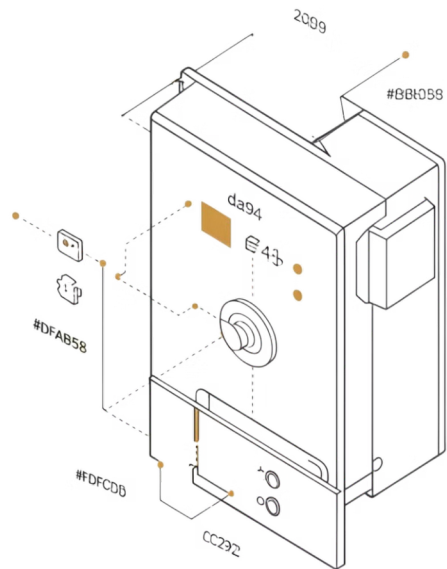
PL/SQL Implementation

```
CREATE OR REPLACE
TRIGGER
  trg_check_ticket_price
BEFORE INSERT OR UPDATE
ON Ticket
FOR EACH ROW
BEGIN
  IF :NEW.price < 0 THEN

    RAISE_APPLICATION_ERROR(
      -20002,
      'Price cannot be
negative.'
    );

  END IF;
END;
```

```
ORA-20001: Error: Minimum ticket price must be at least Rs. 50.
ORA-06512: at "24F-0567.TRG_ENFORCE_MIN_PRICE", line 3
ORA-04088: error during execution of trigger '24F-0567.TRG_ENFORCE_MIN_PRICE'
```



Data Fetching — Procedures

What Are Procedures?

PL/SQL Procedures are reusable, modular code blocks that perform complete transactions. They support **IN**, **OUT**, and **IN OUT** parameters, making them flexible for a wide range of operations.

Example Use Case: Securely inserting a new station into the transit network through a controlled, parameterized interface rather than direct table access.

Using procedures enforces a consistent data entry pattern and allows DBAs to add validation logic in a single centralized location.

PL/SQL Implementation

```
CREATE OR REPLACE PROCEDURE
Add_Station (
  p_id IN NUMBER,
  p_name IN VARCHAR2,
  p_loc IN VARCHAR2,
  p_plat IN VARCHAR2
)
IS
BEGIN
  INSERT INTO Station
VALUES (
  p_id,
  p_name,
  p_loc,
  p_plat
);
COMMIT;
END;
```

```
New passenger added successfully.
-----
          PASSENGER INSERTED
-----
Passenger ID   : 17
Passenger Name : Taheer Bin Yousaf
Email Address  : taheer@email.com
Phone Number   : 03211234567
Date of Birth  : 15-AUG-2002
Calculated Age : 21
-----
Statement processed.
```

Overall System Working & Conclusion



Infrastructure Setup

Administrators define **Stations** and connect them via **Routes** to form the transit network backbone.



Resource Allocation

Vehicles (Buses & Trains) and **Drivers** are registered and managed within the fleet database.



Operations

Schedules are generated, assigning a specific vehicle and driver to a route for a defined timeframe.



User Interaction

Passengers register in the system and purchase **Tickets** linked to specific schedules.



System Integrity

Backend **Triggers** enforce valid data. **Views** and **Procedures** provide streamlined data retrieval for management.

✔ **Project Outcome:** A fully functional, integrity-enforced Smart City Transport database system demonstrating real-world application of advanced PL/SQL concepts — built by Usman Ali Shahid & Taheer Bin Hussain at FAST-NUCES, Faisalabad.